

Universidade Federal de Santa Catarina
Departamento de Informática e Estatística
Modelagem e Simulação – INE5425

Gabriel Salmoria
Lucas Vieira de Souza
Diego Martins Meditsch

Implementação de um Componente Unificado de Análise de Dados no GenESyS

Tema 11.3.1 – Diagnósticos Adicionais e Ajustes de Curvas

Florianópolis, junho de 2026

Sumário

1	Introdução	2
2	Desenvolvimento	3
2.1	Organização do trabalho	3
2.2	Extensão da interface Fitter_if (M1)	3
2.3	Interface DataAnalyzer_if e structs de saída (M2)	4
2.4	Carregamento de dados e estatísticas descritivas (M3)	4
2.5	Ajuste de distribuições (M4)	5
2.6	Testes de aderência (M5 e M6)	5
2.7	Análise de séries temporais (M7)	5
2.8	Inferência paramétrica (M8)	5
2.9	Integração com o simulador	6
3	Validação	7
3.1	Problemas encontrados durante o desenvolvimento	8
4	Conclusão	9

1 Introdução

O pacote `source/tools/` do GenESyS já contava com implementações para ajuste de distribuições (`Fitter_if`), testes de hipóteses (`HypothesisTester_if`) e resolução numérica (`Solver_if`), mas não havia nenhuma interface que integrasse essas capacidades de forma coesa. Quem precisava analisar dados de saída de uma simulação tinha que instanciar e coordenar manualmente esses três componentes, reimplementar cálculos básicos como quartis e histograma, e cuidar da leitura de arquivos por conta própria.

O objetivo deste trabalho foi implementar o componente **DataAnalyzer**: uma fachada que centraliza análise estatística descritiva, ajuste de curvas, testes de aderência, análise de séries temporais e inferência paramétrica em uma única interface, integrada ao mecanismo de traits do simulador. O tema se enquadra na diretriz 11.3.1 da proposta – *foco maior em diagnósticos adicionais e ajustes de curvas*.

O trabalho foi desenvolvido por três estudantes ao longo do semestre. A implementação foi feita incrementalmente, em marcos bem definidos, cada um validado antes de avançar para o próximo. O resultado é um componente funcional, com 26 testes automatizados e pull request direcionado ao branch `2026-1` do repositório.

2 Desenvolvimento

2.1 Organização do trabalho

O trabalho foi dividido em nove marcos (*milestones* M1–M9), cada um com escopo bem delimitado e verificado por compilação e teste antes de avançar. A divisão não foi estritamente por pessoa, mas por dependência técnica: os primeiros marcos estabeleceram as interfaces e extensões necessárias; os seguintes adicionaram funcionalidades sobre essa base; M9 fechou com o harness de testes.

Como artefatos do processo, o grupo manteve três documentos ao longo do desenvolvimento. O primeiro é o `DEVELOPMENT_DataAnalyzer.md`, um diário técnico que registra, para cada marco, o problema que motivou a mudança, o que foi alterado e as decisões de projeto – inclusive as que foram deliberadamente rejeitadas, como o uso de `StatisticsDataFile_if` (discutido na Seção ??). O segundo é a própria suíte de testes do comando `--demo`, que funciona como especificação executável: ao documentar comportamentos esperados em código, ela serve tanto de validação quanto de referência para quem for manter o componente. O terceiro é o pull request sem conflitos direcionado ao branch 2026-1, que inclui todos os arquivos modificados e criados.

A Tabela 1 lista os arquivos afetados.

Tabela 1: Arquivos modificados ou criados

Arquivo	Ação
<code>Fitter_if.h</code>	Adicionado <code>setDataValues()</code>
<code>FitterDefaultImpl.h</code>	Adicionados <code>setDataValues()</code> e <code>_computeStatsFromLoadedData()</code>
<code>FitterDummyImpl.h/.cpp</code>	Stub vazio para satisfazer a interface
<code>DataAnalyzer_if.h</code>	Novo – interface pública e structs de saída
<code>DataAnalyzerDefaultImpl.h</code>	Novo – declaração da classe concreta
<code>DataAnalyzerDefaultImpl1.cpp</code>	Novo – toda a lógica (≈ 810 linhas)
<code>TraitsTools.h</code>	Especialização <code>TraitsTools<DataAnalyzer_if></code>
<code>CMakeLists.txt</code>	<code>DataAnalyzerDefaultImpl1.cpp</code> adicionado às fontes
<code>main.cpp</code>	Comando <code>--demo</code>

Todos os arquivos estão em `source/tools/`.

2.2 Extensão da interface `Fitter_if` (M1)

`FitterDefaultImpl` só aceitava dados via arquivo binário. Para que o `DataAnalyzer` pudesse repassar ao fitter um vetor já em memória, foi preciso adicionar `setDataValues(const std::vector<double>&)` tanto à interface `Fitter_if` quanto à implementação concreta.

A lógica interna de `FitterDefaultImpl` foi ligeiramente refatorada: o bloco que calculava estatísticas de cache foi extraído para o método privado `_computeStatsFromLoadedData()`, chamado agora tanto pelo caminho de arquivo binário quanto pelo novo caminho em memória. Os flags `_cacheLoaded` e `_cacheUsable` são definidos por ambos, de modo que o mecanismo de *lazy loading* em `_ensureDataLoaded()` enxerga os dados como já carregados e não tenta abrir

arquivo algum.

2.3 Interface `DataAnalyzer_if` e structs de saída (M2)

A interface `DataAnalyzer_if` define cinco structs de saída puramente de dados, sem métodos:

- `SummaryStatistics` – n, mín, máx, amplitude, média, mediana, moda, Q1, Q3, variância, desvio-padrão, CV, assimetria, curtose;
- `HistogramData` – número de classes, limites inferiores, frequências absolutas e relativas;
- `BoxplotData` – mín, Q1, mediana, Q3, máx, IIQ, lista de outliers;
- `FitResult` – nome da distribuição, SSE, até cinco parâmetros, flag de validade;
- `GoFResult` – nome, estatística de teste, p-valor, valor crítico, nível de significância, decisão e conclusão em texto.

A grafia americana (*Analyzer*) foi adotada para evitar colisão com o antigo `DataAnalyser_if` (grafia britânica), que possui dependência de `ExperimentManager_if` e pertence a outra camada arquitetural.

2.4 Carregamento de dados e estatísticas descritivas (M3)

O carregamento de dados suporta dois caminhos. Via `setDataValues(vector<double>)`, os dados são armazenados em memória, o vetor é ordenado em `_sortedData` e as estatísticas de cache são computadas. Via `setDataFilename`, o parser `SimulationResultsDatasetParser::loadFromTextFile` converte arquivos nos quatro formatos texto do GenESyS (`RawNumeric`, `RecordLegacy`, `RecordEnriched`, `GuiTabular`) e chama `setDataValues` em seguida.

Os quantis são calculados pelo método R7 (interpolação linear): dado percentil p , calcula-se $h = (n - 1)p$, $i = \lfloor h \rfloor$, $f = h - i$, e o resultado é $x_{(i)}(1 - f) + x_{(i+1)}f$. Assimetria e curtose são calculadas como momentos padronizados de terceira e quarta ordem, com curtose em excesso (subtraindo 3).

Decisão de projeto: `StatisticsDataFile_if`

A proposta sugeria encaminhar as estatísticas descritivas pela interface `StatisticsDatafile_if`. Essa interface estende `CollectorDatafile_if`, que é estruturalmente acoplada a um arquivo binário de coletor – não há construtor nem setter que aceite `vector<double>`. Utilizá-la exigiria escrever os dados em arquivo binário e relê-los, derrotando o propósito de trabalhar em memória. A decisão foi calcular as estatísticas diretamente, produzindo resultados equivalentes sem o custo de I/O adicional.

2.5 Ajuste de distribuições (M4)

O método `fitDistribution(nome)` despacha para o método correto de `FitterDefaultImpl` com base no nome e empacota SSE e parâmetros em `FitResult`. São suportadas sete distribuições: uniforme, triangular, normal, exponencial, erlang, beta e Weibull.

O método `fitAll()` chama `_fitter.fitAll()` para obter o nome da melhor distribuição e, em seguida, chama `fitDistribution(nome)` para recuperar os parâmetros. O duplo ajuste é intencional: `fitAll` retorna apenas nome e SSE, não os parâmetros.

2.6 Testes de aderência (M5 e M6)

Para o qui-quadrado, o número de classes segue a fórmula de Sturges ($k = \max(5, 1 + \lceil \log_2 n \rceil)$). Classes com frequência esperada $E < 5$ são fundidas com a adjacente. O grau de liberdade é $k_{\text{fundido}} - 1 - p$, onde p é o número de parâmetros estimados a partir dos dados. O p-valor é obtido integrando numericamente a PDF da qui-quadrado via `SolverDefaultImpl1`; o valor crítico por bissecção sobre a CDF acumulada.

Para o Kolmogorov-Smirnov, a estatística $D_n = \max_i |F_n(x_i) - F(x_i)|$ é calculada sobre as estatísticas de ordem. O p-valor usa a série assintótica de Kolmogorov com 200 termos; o valor crítico usa a fórmula aproximada $D_\alpha \approx \sqrt{-\ln(\alpha/2)/(2n)}$. A aproximação assintótica subestima levemente o p-valor para $n < 35$, o que é aceitável para o contexto.

2.7 Análise de séries temporais (M7)

A média móvel usa janela deslizante com soma acumulada em $O(n)$, produzindo $n - w + 1$ valores. A autocorrelação usa o estimador enviesado $\hat{r}(k) = \hat{c}(k)/\hat{c}(0)$, com `maxLag` silenciosamente limitado a $n - 1$ para evitar leitura fora dos limites. O correlograma retorna o mesmo cálculo que a autocorrelação, distinguindo-se apenas conceitualmente – sua intenção é ser renderizado como gráfico de barras por defasagem.

2.8 Inferência paramétrica (M8)

Os métodos de IC e teste de hipóteses delegam para `HypothesisTesterDefaultImpl1`, usando as estatísticas em cache. Os testes para duas populações requerem que `loadSecondSample()` tenha sido chamado previamente.

Uma versão inicial de `testProportionTwoSamples` e `testVarianceTwoSamples` invocava por engano as sobrecargas de uma população do `HypothesisTesterDefaultImpl1`, tratando a estatística da segunda amostra como valor da hipótese nula. A correção foi passar `_count2` como quarto argumento, selecionando as sobrecargas corretas de duas populações, que já existiam na implementação.

2.9 Integração com o simulador

O componente é registrado no mecanismo `TraitsTools<T>` do GenESyS:

```
1 template <> struct TraitsTools<DataAnalyzer_if> {  
2     typedef DataAnalyzerDefaultImpl1 Implementation;  
3 };
```

Listing 1: Registro em TraitsTools.h

Qualquer módulo do simulador pode obter uma instância concreta via `TraitsTools<DataAnalyzer_if>` sem depender da classe concreta diretamente. O executável `genesys_tools_application` foi estendido com o comando `--demo`, descrito na seção de validação.

3 Validação

A validação foi integrada ao próprio executável como o comando `--demo`, que instancia `DataAnalyzerDefaultImpl1` diretamente e executa 26 verificações organizadas em sete seções. Cada verificação imprime `[PASS]` ou `[FAIL]`. Três conjuntos de dados fixos são usados: 50 valores amostrados de $N(5, 1)$, 20 valores de $\text{Exp}(2)$ e 20 valores com tendência crescente de 1,1 a 3,0.

Carregamento de dados. Verifica que `setDataValues` aceita 50 valores, que `setDataFilename` carrega os mesmos 50 valores de um arquivo `RawNumeric` temporário, que `clearData` esvazia o conjunto e que `loadSecondSample` armazena um segundo vetor.

Estatísticas descritivas. Verifica que a média está em $[4,5; 5,5]$ para dados $N(5, 1)$, que o ordenamento $\text{mín} < Q_1 < \text{mediana} < Q_3 < \text{máx}$ se mantém, que o desvio-padrão é positivo, e que `quartile(2)`, `decile(5)` e `centile(50)` retornam o mesmo valor (todos são a mediana).

Histograma e boxplot. Verifica que a soma das frequências absolutas é $n = 50$, que o histograma tem exatamente 8 classes, que as frequências relativas somam 1,0, e que $\text{IIQ} = Q_3 - Q_1$.

Ajuste de distribuições. Verifica que o ajuste normal é válido e com média em $[4,5; 5,5]$, que `fitAll` retorna resultado válido com SSE menor ou igual ao do ajuste normal, e que a média do ajuste exponencial nos dados $\text{Exp}(2)$ fica em $[1,0; 3,5]$.

Testes de aderência. Verifica que nem o qui-quadrado nem o KS rejeitam o ajuste normal a $\alpha = 0,05$ quando os dados são de fato normais.

Séries temporais. Verifica que `movingAverage(3)` produz $n - w + 1 = 18$ valores, que o primeiro valor é a média dos três primeiros elementos, que `autocorrelation(5)` retorna 6 valores com $\hat{r}(0) = 1$ e $\hat{r}(1) > 0,5$ para dados com tendência, e que `correlogram(5)` retorna o mesmo que `autocorrelation(5)`.

Inferência paramétrica. Verifica que o IC 95% para a média contém o valor verdadeiro 5,0, que o teste t não rejeita $\mu_0 = 5,2$ (próximo à média amostral de 5,206) mas rejeita $\mu_0 = 10,0$, que o IC de variância tem limite superior maior que o inferior, e que o teste de médias para duas populações rejeita $H_0 : \mu_1 = \mu_2$ ao comparar dados normais com exponenciais.

Todos os 26 testes passam com a implementação atual. A saída completa do comando é:

```
=====
DataAnalyzer Demo -- GenESyS tools
=====
1. DATA LOADING
```



```

-----
[PASS] setDataValues accepted 50 values
[PASS] setDataFilename loaded same 50 values from RawNumeric text file
[PASS] clearData empties the dataset
[PASS] loadSecondSample stores a second vector
...
=====
Demo complete.
=====

```

3.1 Problemas encontrados durante o desenvolvimento

Três problemas foram encontrados e corrigidos antes da versão final.

O primeiro foi uma incompatibilidade de tipos no CDF de Erlang: `std::llround` retorna `long long int`, enquanto a literal `1L` é `long int`. O compilador recusou `std::max(1L, std::llround(p2))` por tipos conflitantes; a correção foi usar `1LL`.

O segundo envolveu variáveis de intervalo de confiança declaradas como `const auto`: os métodos `inferiorLimit()` e `superiorLimit()` de `HypothesisTester_if::ConfidenceInterval` não são declarados `const`, então a qualificação `const` na variável local impedia a chamada. A correção foi declarar as variáveis CI simplesmente como `auto`.

O terceiro foi o bug nas sobrecargas dos testes de duas populações, descrito na Seção 2.8. A correção foi detectada ao revisar os testes e constatar que o segundo conjunto de dados não estava influenciando o resultado do teste (detalhes na Seção 2.8).

4 Conclusão

O componente `DataAnalyzer` foi implementado e integrado ao GenESyS, atendendo à diretriz 11.3.1 da proposta. Partindo da necessidade de coordenar manualmente `Fitter_if`, `HypothesisTester_if` e parsers de arquivo, o grupo construiu uma fachada única que cobre análise descritiva, ajuste de sete distribuições com seleção automática, dois testes de aderência (qui-quadrado e KS), média móvel e autocorrelação, e inferência paramétrica para uma e duas populações.

A decisão de organizar o desenvolvimento em marcos com documentação contínua se mostrou útil: o diário técnico em `DEVELOPMENT_DataAnalyzer.md` registrou as motivações por trás de cada mudança – inclusive a decisão de não usar `StatisticsDataFile_if` – e facilitou a revisão entre os membros. O comando `--demo` com 26 testes automatizados garantiu que nenhum marco posterior quebrou os anteriores.

O componente está pronto para ser consumido por outros módulos do simulador via `TraitsTools<DataAnalyzer_if>::Implementation`, e serve de base direta para futuros painéis gráficos de análise de resultados.

Referências